COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-031 / Visualizations and Data Analysis	**SECTION:**	CPE21S1
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 1, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Sir Jimlord Quejado

HOA 10.1 Graphs Hands-On Activity Graphs

1. PROCEDURE OUTPUT

ILO A

```cpp
#ifndef CPE010_HOA10P1_TORRES_SC2_H
#define CPE010_HOA10P1_TORRES_SC2_H

#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        this->N = N;
        // allocate array of pointers and initialize to nullptr
        head = new adjNode*[N]();

        // construct directed graph by adding edges to it
        for (int i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor - Fixed to properly clear linked lists
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* curr = head[i];
            while (curr != nullptr) {
                adjNode* nextNode = curr->next;
                delete curr; // Delete individual node
                curr = nextNode;
```

```
60              }
61          }
62          delete[] head; // Delete the array of pointers
63      }
64  };
65
66  // print all adjacent vertices of given vertex
67  void display_AdjList(adjNode* ptr, int i) {
68      while (ptr != nullptr) {
69          std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
70          ptr = ptr->next;
71      }
72      std::cout << std::endl;
73  }
74
75  #endif // CPE010_HOA10P1_TORRES_SC2_H
```

CODE CONSOLE HEADER FILE

```
#include <iostream>
#include "CPE010_HOA10p1_TORRES_sc2.h"

int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0, 1, 2}, {0, 2, 4}, {1, 4, 3}, {2, 3, 2}, {3, 1, 4}, {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

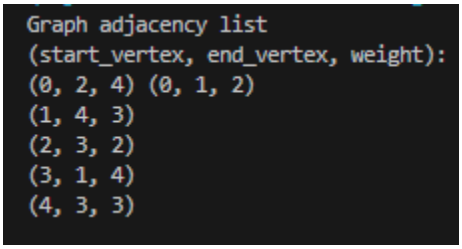
    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list \n(start_vertex, end_vertex, weight):" << std::endl;

    // Fixed: Cleaned up scope braces and enclosed the loop properly
    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

CODE CONSOLE MAIN FILE



OUTPUT CONSOLE

Code and Observation Analysis	**In the ILO A, I've built a directed graph using an array of linked list heads to represent an adjacency list. From my error perspective, I need proper memory management which requires ~DiaGraph() to iteratively delete each dynamic adjNode to prevent memory leaks, alongside enclosing loop bodies in main() with scope braces for correct iteration. It is because the new edges are added to the head of each vertex list, which the console outputs connected vertices in reverse-insertion (LIFO) order.**

ILO B

```cpp
#ifndef CPE010_HOA10P1_TORRES_SC4_H
#define CPE010_HOA10P1_TORRES_SC4_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (size_t i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const { return V; }
    auto &edges() const { return edge_list; }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
```

```
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)    variable "size_t" is not a type name
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    // Fixed: Properly explicit keyed map indexing
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
void test_DFS()
{
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

#endif // CPE010_HOA10P1_TORRES_SC4_H
```

CODE CONSOLE HEADER FILE

```cpp
#include <iostream>
#include "CPE010_HOA10p1_TORRES_sc4.h"

int main()
{
using T = unsigned;
test_DFS<T>();
return 0;
}

```

CODE CONSOLE MAIN FILE

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
9:

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

OUTPUT CONSOLE

Code and Observation Analysis	In the ILO B code, I've executed the **Depth-First Search (DFS) on a reference graph using a std::stack and std::set to track visited vertices**. I've noticed that there **are corrections that were required for explicit map key indexing when populating edges in create_reference_graph()**, alongside resolving **IDE template type warnings regarding size_t**. By utilizing the LIFO stack mechanics and structure, the **algorithm traverses along each branch as deeply as possible before backtracking to unvisited nodes**.

ILO C

```cpp
#ifndef CPE010_HOA10P1_TORRES_SC6_H
#define CPE010_HOA10P1_TORRES_SC6_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    // Fixed: Loop bound corrected to '<=' to display the 9th vertex
    for (size_t i = 1; i <= G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os; // Fixed: Added missing return statement
}

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }
    auto &edges() const
    {
        return edge_list;
    }
```

```
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Fixed: Changed template parameter to 'U' to avoid shadowing class template parameter 'T'
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    // Fixed: Standard map syntax assignments map perfectly here
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
```

```
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)    variable "size_t" is not a type name
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                // Optimization: Only push if it hasn't been visited yet
                if (visited.find(e.dest) == visited.end())
                {
                    queue.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

#endif // CPE010_HOA10P1_TORRES_SC6_H
```

CODE CONSOLE HEADER FILE

```cpp
#include <iostream>
#include "CPE010_HOA10p1_TORRES_sc6.h"

int main()
{
    using T = unsigned;
        test_BFS<T>();
return 0;
}
```

CODE CONSOLE MAIN FILE

```
      {2: 2}, {5: 3},
      {1: 2}, {5: 5}, {4: 1},
      {4: 2}, {7: 3},
      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
      {1: 3}, {2: 5}, {4: 2}, {8: 3},
      {4: 4}, {7: 4}, {8: 1},
      {3: 3}, {6: 4},
      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

OUTPUT CONSOLE

Code and Observation Analysis	In the ILO C code, **I've implemented the Breadth-First Search (BFS) using a std::queue data structure to discover adjacent nodes level-by-level**. Moreover, an **optimization check ensures the neighbor vertices are only pushed into the queue if they have not yet been marked as visited**. I've **noticed some typo errors but it still work for the size_t name variable**.

2. SUPPLEMENTARY OUTPUT

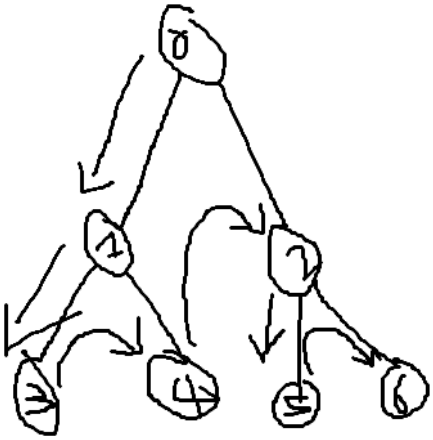
ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

If its backtracking, I would recommend the **DEPTH FIRST SEARCH because it implements a stack structure which the structure traverse along each branch as deeply as possible before backtracking to unvisited nodes**.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

GRAPHICAL COMPARISON

TREE DFS	GRAPH DFS
	

PSEUDOCODE

TREE DFS	GRAPH DFS

FUNCTION TreeDFS(node)
** IF node IS NULL THEN**
** RETURN**
** END IF**
 	
** PROCESS(node) // e.g., print or evaluate node data**
 	
** TreeDFS(node.left) // Recurse left child**
** TreeDFS(node.right) // Recurse right child**
END FUNCTION
	FUNCTION GraphDFSDriver(graph)
	** INITIALIZE visited AS Set/Array (all set to FALSE)**
	
	** FOR EACH vertex IN graph.vertices DO**
	** IF vertex NOT IN visited THEN**
	** GraphDFSVisit(graph, vertex, visited)**
	** END IF**
	** END FOR**
	END FUNCTION
	
	FUNCTION GraphDFSVisit(graph, vertex,

	visited) ADD vertex TO visited // Prevent infinite looping on cycles PROCESS(vertex) // e.g., print or evaluate vertex data FOR EACH neighbor IN graph.adjacencyList[vertex] DO IF neighbor NOT IN visited THEN GraphDFSVisit(graph, neighbor, visited) END IF END FOR END FUNCTION

CODE IMPLEMENTATION

TREE DFS	GRAPH DFS

#include <iostream>
 	
// Node definition for a Binary Tree
struct TreeNode {
  int data;
  TreeNode* left;
  TreeNode* right;
 	
  TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};
 	
// Tree DFS (Preorder Traversal Example)
void treeDFS(TreeNode* root) {
  // Base Case: Clean exit when a null child is reached
  if (root == nullptr) {
  return;
  }
 	
  // Process the current node
  std::cout << root->data << " ";
 	
  // Recursively traverse structural children without any loop safety checks
  treeDFS(root->left);
  treeDFS(root->right);
}
	#include <iostream>
	#include <vector>
	
	class Graph {
	  int vertices;
	  std::vector<std::vector<int>> adjList; // Adjacency list representation
	
	  // Internal helper function for recursive DFS execution
	  void dfsUtil(int vertex, std::vector<bool>& visited) {
	  // Mark the current vertex as visited to prevent cycle loops
	  visited[vertex] = true;
	  std::cout << vertex << " ";
	
	  // Traverse all adjacent neighbors of the current vertex
	  for (int neighbor : adjList[vertex]) {
	  if (!visited[neighbor]) {
	  dfsUtil(neighbor, visited);
	  }
	  }
	  }
	
	public:
	  Graph(int v) : vertices(v), adjList(v) {}
	
	  void addEdge(int u, int v) {
	  adjList[u].push_back(v); // Assuming a directed graph edge
	  }
	
	  // Wrapper function handling

	disconnected graph elements **void graphDFS() {** **std::vector<bool> visited(vertices, false);** **// Loop ensures disconnected components are fully traversed** **for (int i = 0; i < vertices; ++i) {** **if (!visited[i]) {** **dfsUtil(i, visited);** **}** **}** **}** **};**

3. In the performed code, what data structure is used to implement the Breadth First Search?

I think its Queues because in the ILO C a std::queue data structure is needed to discover adjacent nodes level-by-level. Moreover, an **optimization check ensures the neighbor vertices are only pushed into the queue if they have not yet been marked as visited**.

4. How many times can a node be visited in the BFS?
In standard Breadth-First Search (BFS), a node is visited and processed only once.

CONCLUSIONS/LEARNINGS/ANALYSIS	This **CPE010 Hands On laboratory activity successfully demonstrates graph representation through adjacency lists and highlights the structural distinctions between Depth-First Search (DFS) using a LIFO stack and Breadth-First Search (BFS) using a FIFO queue**, **emphasizing the necessity of a `visited` set to prevent infinite cycles in cyclic graph topologies compared to acyclic tree traversals like Pre-Order traversal**. From my point-of-view debugging, **there are some critical corrections included preventing memory leaks in ILO A by updating `~DiaGraph()` to explicitly free dynamic linked list nodes, fixing map key overwrites and template type warnings in ILO B, and correcting loop bounds, missing stream returns, and friend template shadowing in ILO C**. Overall, these implementations helps me improve my essential competencies in **C++ dynamic memory management, pointer allocation techniques, and graph traversal optimization.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: NONE

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: NONE

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.